BFS — صياغة المسألة

المطلوب إيجاد طريق من A إلى G في شجرة المثال. تكلفة كل وصلة 1. نستخدم BFS لأنها تبحث مستوى بمستوى، فتصل إلى أقرب هدف بعدد الوصلات.

Problem Formulation

1. **State space:** الحالات الموجودة في الجراف، من A إلى G.
2. **Initial state:** البداية A.
3. **Actions:** الانتقال إلى جار مباشر في اتجاه السهم.
4. **Transition model:** اختيار جار يجعل هذا الجار هو الحالة الجديدة.
5. **Goal test:** هل الحالة الحالية تساوي G؟
6. **Path cost:** مجموع تكاليف الوصلات؛ هنا يساوي عددها.

الفكرة في أربع خطوات

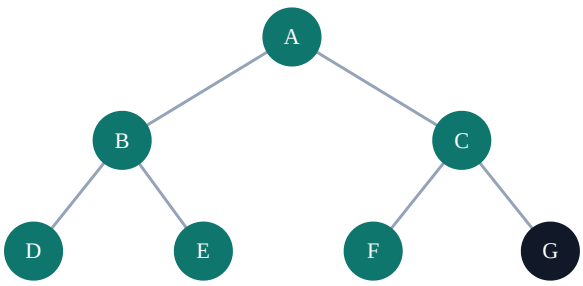
1. نضع البداية في Queue ونسجّلها في visited.
2. نخرج أول حالة ونفحص هل هي الهدف.
3. إذا لم تكن الهدف نضيف جيرانها غير المكتشفين إلى نهاية الطابور.
4. نكرر حتى الوصول للهدف أو انتهاء الطابور.

نتيجة المثال

```
Visit order: A → B → C → D → E → F → G
Solution:    A → C → G
Path cost:   2
```

ترتيب الزيارة هو ترتيب الفحص، وليس مسار الحل. الحل نستخرجه من روابط الآباء المخزّنة في parent.

شجرة المثال



البداية A والهدف G، والجيران من اليسار إلى اليمين، مثل صورة السكشن.

الخوارزمية كاملة — bfs.py

ابدأ بقراءة الطابور وvisited، ثم parent، واترك steps للنهاية لأنها تخص العرض فقط.

```
"""BFS: Queue + visited + parent. Each edge costs 1."""
from collections import deque


def bfs(graph, start, goal):
    queue = deque([start])
    visited = {start}          # Mark a state when it enters the queue.
    parent = {start: None}     # Remember how we reached each state.
    order = []
    steps = []

    while queue:
        current = queue.popleft()
        order.append(current)

        # Follow parent links to build the route to the current state.
        path = []
        node = current
        while node is not None:
            path.append(node)
            node = parent[node]
        path.reverse()

        if current != goal:
            for neighbor in graph[current]:
                if neighbor not in visited:
                    visited.add(neighbor)
                    parent[neighbor] = current
                    queue.append(neighbor)

        # Copies keep earlier animation steps unchanged.
        steps.append({
            "current": current,
            "queue": list(queue),
            "order": order.copy(),
            "path": path,
        })

        if current == goal:
            return path, steps

    return [], steps          # No path to the goal.
```

التشغيل ونقاط المناقشة

الملفات التي تحتاج شرحها

bfs.py: البحث وحفظ اللقطات. **app.py:** الجراف، والواجهة، وصياغة المسألة، ودالة show_step للرسم. لا توجد كلاسات أو ملفات مساعدة إضافية.

التشغيل

استخدم Python 3.11 أو أحدث داخل مجلد المشروع:

```
python -m pip install -r requirements.txt
python -m streamlit run app.py
```

اختر البداية والهدف. شريط الخطوات يعرض لقطة واحدة، وزر التشغيل يعرض الأنيميشن من البداية للنهاية. لتغيير الجراف عدّل القاموس في أول app.py.

أسئلة متوقعة

لماذا deque؟ تدعم إخراج أول عنصر بـpopleft في O(1)، وتحقق ترتيب FIFO.

متى نسجل الحالة في visited؟ وقت إضافتها للطابور، لمنع دخولها مرتين أو إعادة الدوران. الاسم هنا يعني «اكتُشفت»، وليس بالضرورة «فُحصت».

لماذا parent؟ لمعرفة الطريق الذي وصلنا به لكل حالة. نتتبع ا الآباء للخلف ثم نعكس القائمة.

لماذا copy في steps؟ كي لا تتغير اللقطات القديمة عندما تتغير قوائم البحث.

لو البداية هي الهدف؟ مسار من حالة واحدة وتكلفة صفر. ولو الهدف غير متاح يفرغ الطابور وتعود قائمة مسار فارغة.

Time & Space — كما في السكشن

```
Time Complexity:  O(b^d)
Space Complexity: O(b^d)
```

هذا تحليل شجرة البحث الوارد في ملف السكشن. b عدد الفروع وd عمق أقرب هدف. في المثال الحالي b = 2 وd = 2، لأن G تقع على بعد وصلتين من A.

هذه حدود نظرية وليست عدد الحالات التي زارتها الخوارزمية. تخزين لقطات الأنيميشن وبناء المسارات تكلفة إضافية للعرض، خارج الحدود المذكورة للبحث.

BFS تضمن أقل عدد وصلات. هذا يساوي أقل تكلفة هنا لأن كل الوصلات تكلفتها 1؛ اختلاف الأوزان يلغي ضمان أقل تكلفة.

المراجع: صورة BFS Worked Example وBFS&DFS section.pdf، وProblem_Formulation.pdf المرفقة.